

第八次实验报告

2026 年 6 月 7 日

1 实验一 流水线 CPU 设计实验

1.1 原理图

流水线 CPU 由五个流水段组成：取指（IF）、译码（ID）、执行（EX）、访存（M）、写回（WB），各段之间通过流水段寄存器传递数据与控制信号。整体原理图参见讲义图 8-1，各子模块引脚布局参见讲义图 8.2—8.5。

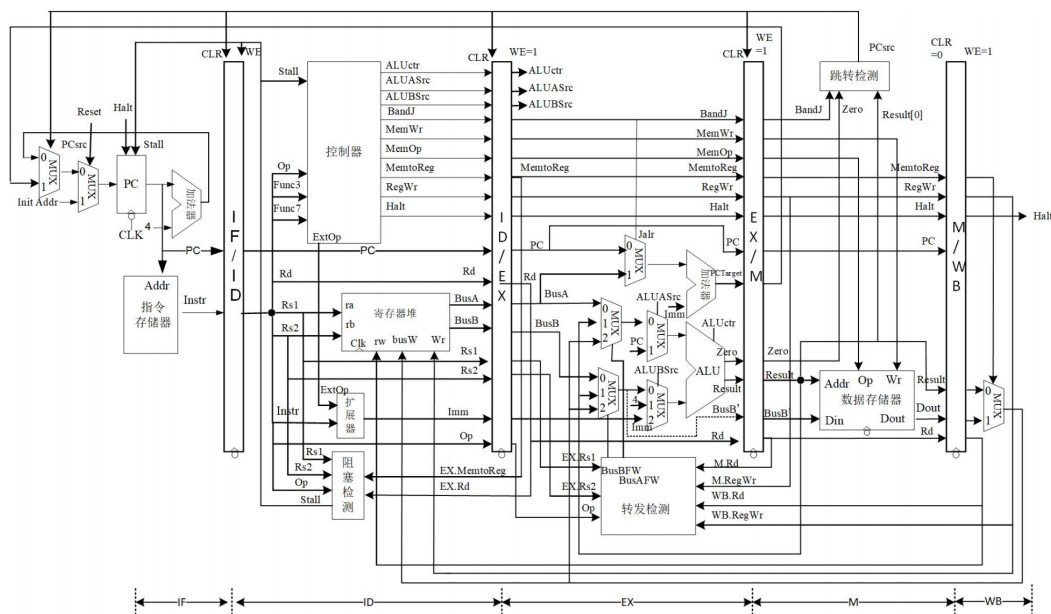


图 1: 流水线 CPU 整体原理图

1.2 电路图

各类子模块的单周期版本在之前的实验中已经展示，这里仅展示流水线 CPU 中新增的关键子模块及顶层电路图。

1.2.1 转发检测模块

转发检测在 EX 段生成转发选择信号 BusAFW / BusBFW。当 EX 段源寄存器号与 M 段（M.rd）或 WB 段（WB.rd）的目的寄存器号相同，且目的寄存器非 x0，且对应指令写寄存器（RegWr=1）时，生成非零转发信号；M 段优先级高于 WB 段。

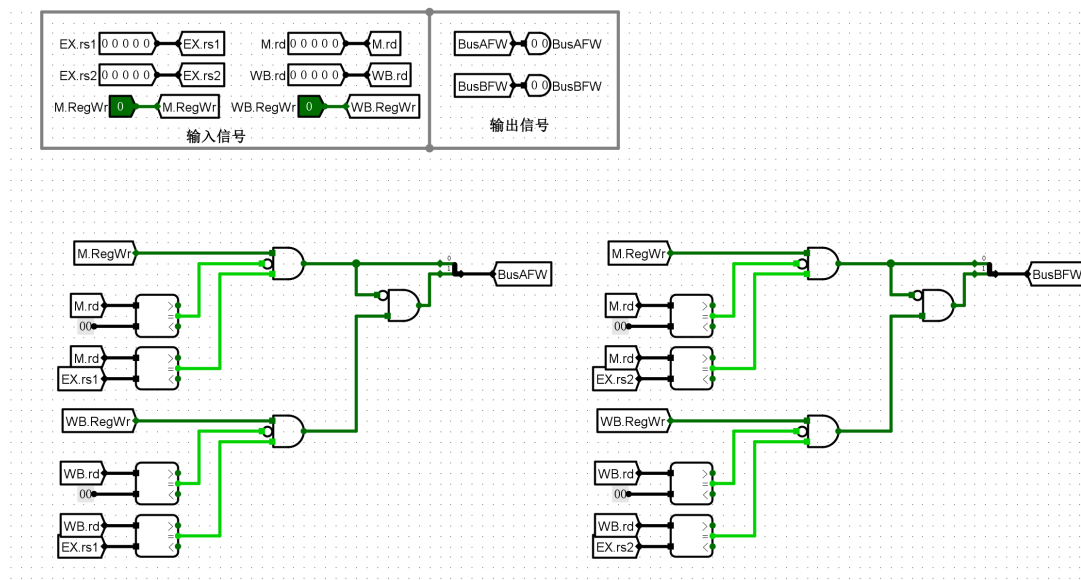


图 2: 转发检测模块电路图

1.2.2 阻塞检测模块

阻塞检测在 ID 段执行，用于解决 load-use 冒险。当前序指令为 load 类型（EX.MemtoReg=1），且其目的寄存器 EX.rd 与当前 ID 段指令的 rs1 或 rs2 相同（需通过 opcode 判断当前指令是否使用 rs1/rs2）时，输出阻塞信号 Stall。

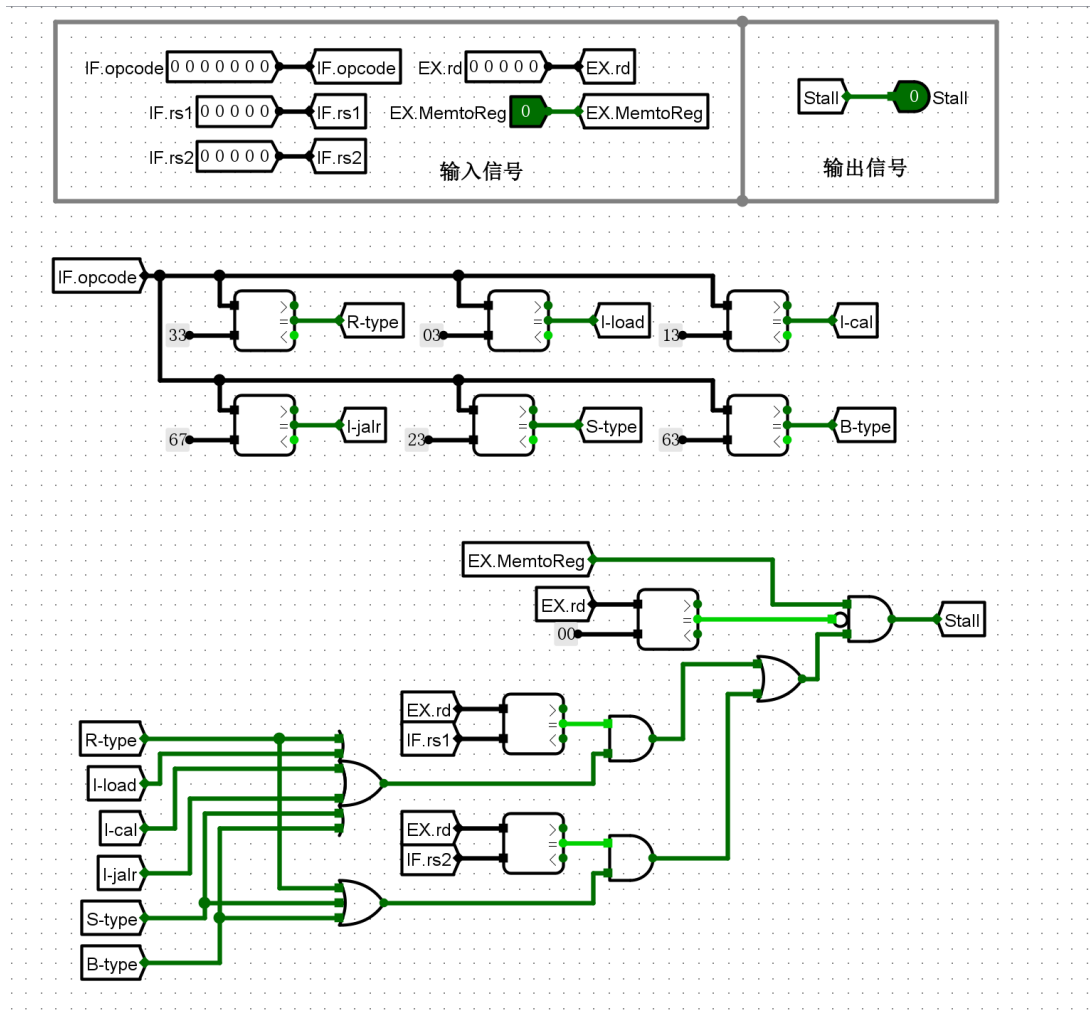


图 3: 阻塞检测模块电路图

1.2.3 跳转检测模块

跳转检测模块（实验一）位于 M 段，输入为 M.BandJ、M.Zero 和 M.Result[0]，输出跳转信号 PCsrc（高电平有效）。PCsrc=1 时，流水线冲刷 IF/ID、ID/EX、EX/M 三个流水段寄存器，PC 跳转至 PCTarget。

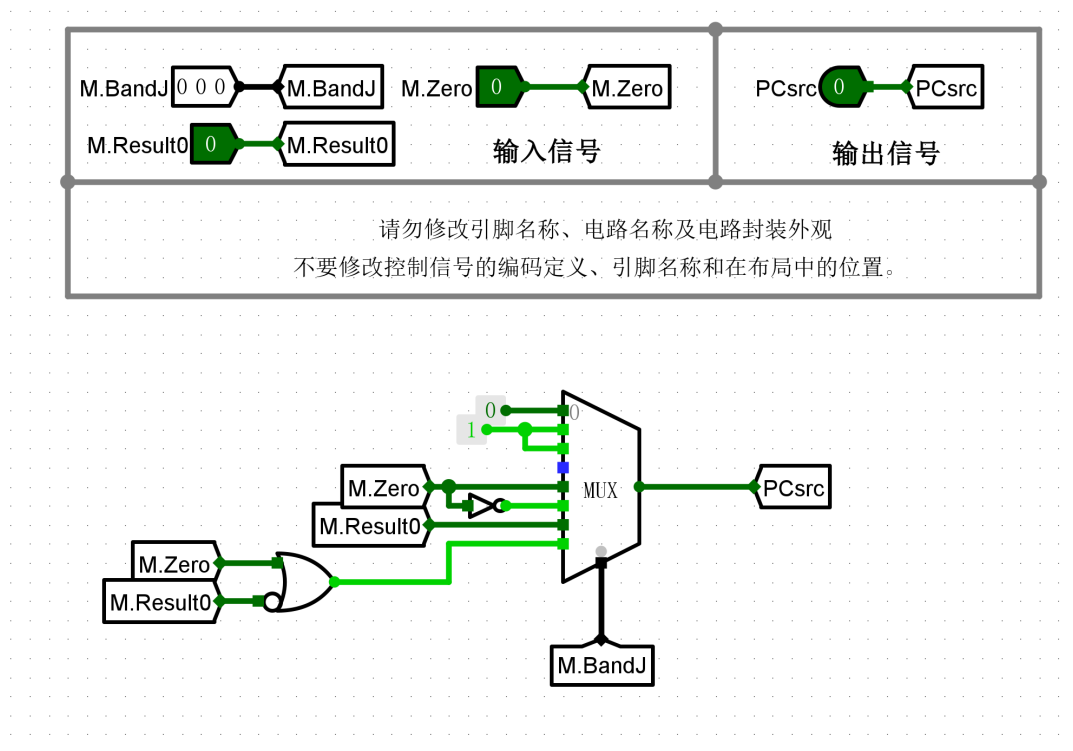


图 4: 跳转检测模块电路图（位于 M 段）

1.2.4 流水段寄存器

流水段寄存器包含时钟 CLK、清零 CLR、写使能 WE 三个控制端，在时钟下降沿写入数据。由于 Logisim 寄存器清零为异步实现，为避免毛刺，本设计通过将输入 MUX 到全零后写入的方式实现**同步清零**：CLR 有效时，WE 同时有效，输入数据强制为 0。

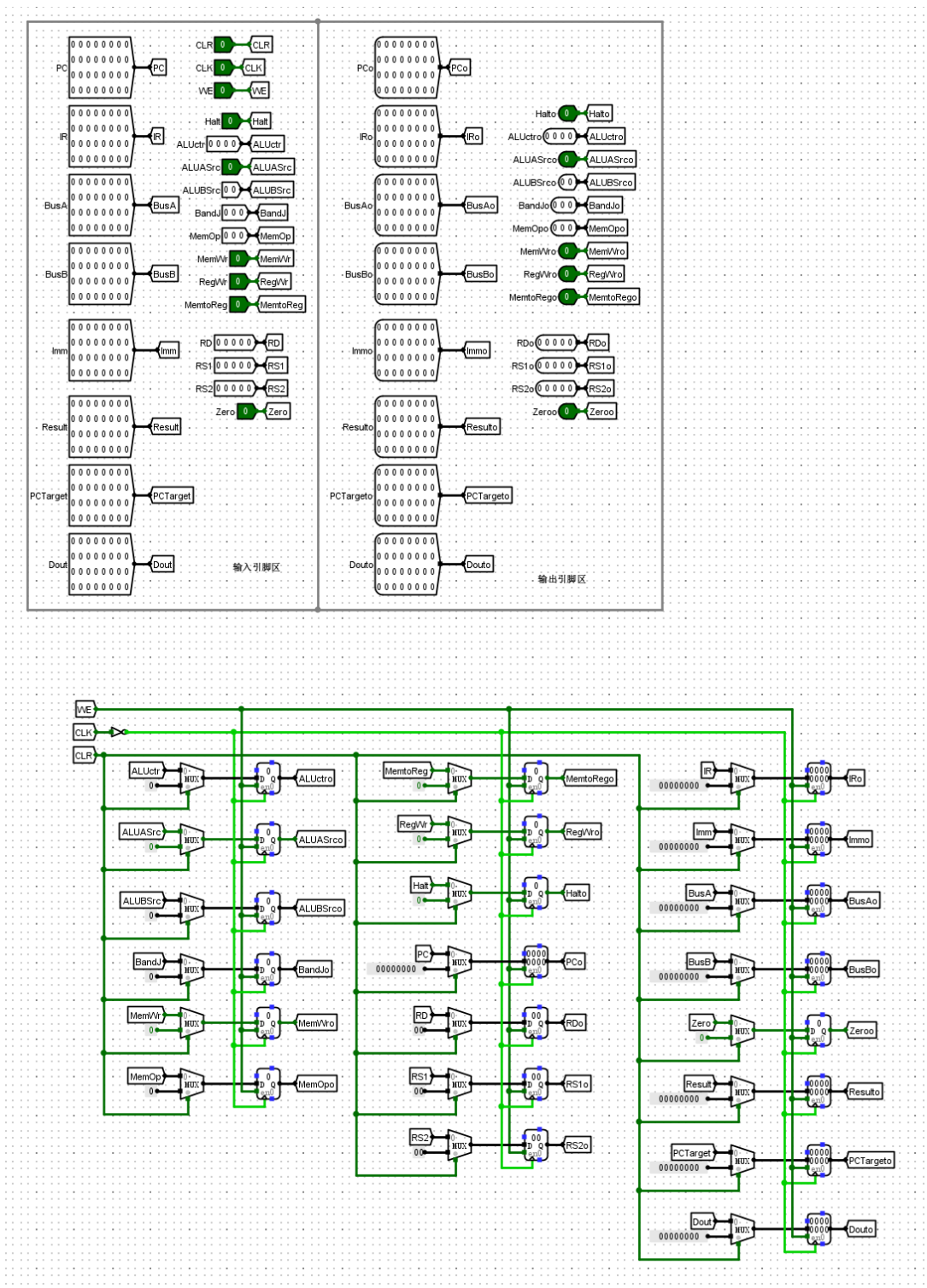


图 5: 流水段寄存器电路图（以 IF/ID 为例）

1.2.5 EU 模块

在 EU 模块中对 ALU 以及操作数选择进行了一定程度的封装，便于简化主电路的布局。

1.3 电路功能

本实验在单周期 RV32I CPU 基础上实现了经典五段流水线处理器，支持全部 RV32I 指令集，并通过以下机制处理三类流水线冒险：

旁路转发 (Data Forwarding) 在 EX 段进行转发检测，将 M 段或 WB 段已产生的结果直接旁路到 ALU 输入端，消除运算类指令之间的 RAW 数据冒险：

- BusAFW / BusBFW = 00：使用寄存器堆输出（无转发）；
- BusAFW / BusBFW = 01：转发来自 EX/M 寄存器的 M.Result；
- BusAFW / BusBFW = 10：转发来自 M/WB 寄存器的 WB.Result（经 MemtoReg MUX 选择后）。

当 M.rd 与 WB.rd 均匹配时，M 段（更新）数据优先级更高。

Load-use 阻塞 (Stall) 在 ID 段进行阻塞检测。当前序 load 指令 (EX.MemtoReg=1) 的目的寄存器与当前指令的 rs1 或 rs2 相同（且非 x0）时，产生 Stall 信号：PC 和 IF/ID 寄存器写使能置无效（内容保持不变），同时将 ID 段所有控制信号清零（插入气泡），其余流水段正常推进。经过一个时钟周期后，load 指令进入 WB 段，后续指令可通过 WB→EX 转发取得数据。

静态分支预测与冲刷 (Flush) 采用“预测不跳转”静态策略，流水线始终顺序取指。在 M 段进行跳转检测，若实际需要跳转 (PCsrc=1)，则将 IF/ID、ID/EX、EX/M 三个流水段寄存器同步清零，PC 跳转至 PCTarget，引入 3 个时钟周期的气泡代价。

1.4 仿真模拟

在指令存储器中加载 lab8.1.hex，按时钟单步或连续执行，观测各流水段 PC、寄存器值及性能计数器输出。测试程序覆盖加减运算、移位、访存、跳转等指令，并专门设计了 RAW 冒险 (test_2 中 add 后立即使用) 和 load-use 冒险 (test_8 中 lb/lw 后立即使用) 的测试用例。若全部测试通过，10 号寄存器输出 0x00c0ffee；否则输出 0xdeaddead 并在 3 号寄存器写入失败测试序号。

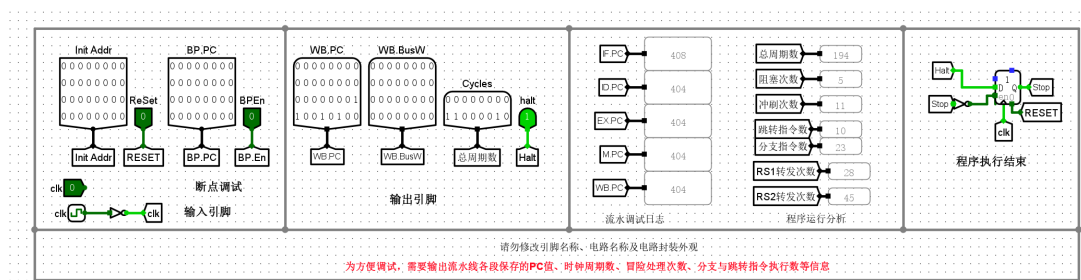


图 8: lab8.1 测试程序仿真结果

仿真结果如下（请根据实际仿真结果填写）：

- 3 号寄存器输出值：0x0000000e
- 10 号寄存器输出值：0x00c0ffee
- 12 号寄存器输出值：0x00000059
- 总周期数：194，阻塞次数：5，冲刷次数：11
- 跳转指令执行数：10，分支指令执行数：23
- RS1 转发次数：28，RS2 转发次数：45

1.5 错误现象及分析

在完成本次实验的过程中，遇到了以下问题：

1. **转发 MUX 连线错误：**在连接 BusAFW / BusBFW 多路选择器时，误将 WB 段转发数据 (WB.Result) 和 M 段转发数据 (M.Result) 的输入顺序接反，导致 RAW 冒险测试用例 (test_2 的 add 指令) 输出错误结果。检查后发现连线顺序与转发逻辑定义 (BusAFW=01 对应 M.Result, BusAFW=10 对应 WB.Result) 不符，修正连线后测试通过。
2. **寄存器堆时钟边沿设置错误：**没有正确将寄存器堆 clk 信号取反导致写入边沿也为下降沿，在 load-use 冒险时，设计中的转发两条指令不能正确解决该冒险。在加上取反后，寄存器堆正确设置为上升沿写入，问题解决。

2 实验二 测试程序优化验证

2.1 原始累加和程序 (lab8.2)

原始累加和程序 lab8.2.asm 的汇编代码如下：

Listing 1: 累加和原始程序 lab8.2.asm

```
1 .text
2 main:
3     lw    a0, 0(x0)          # 从数据区 0x0 读取参数 n; n=0 则退出
4     beq   a0, x0, fail
5     ori   a2, x0, 1          # 循环变量 i = 1
6     xor   a3, a3, a3          # 累加和 S = 0
7 loop:
8     add   a3, a3, a2          # S = S + i
```



```
9      bgeu a2, a0, finish  # i >= n 则跳出循环
10     addi a2, a2, 1      # i++
11     jal  x0, loop        # 无条件跳转（必然冲刷 3 周期）
12  finish:
13     sw    a3, 4(x0)      # 保存结果到 Mem[0x4]
14  pass:
15     lui   a0, 0xc10
16     addi a0, a0, -18     # a0 = 0x00c0ffee
17     ecall
18  fail:
19     lui   a0, 0xdeade
20     addi a0, a0, -339    # a0 = 0xdeaddead
21     ecall
```

在数据存储器地址 0x0 处设置参数 $n = 100$ （即 0x64），执行程序，仿真结果如下：

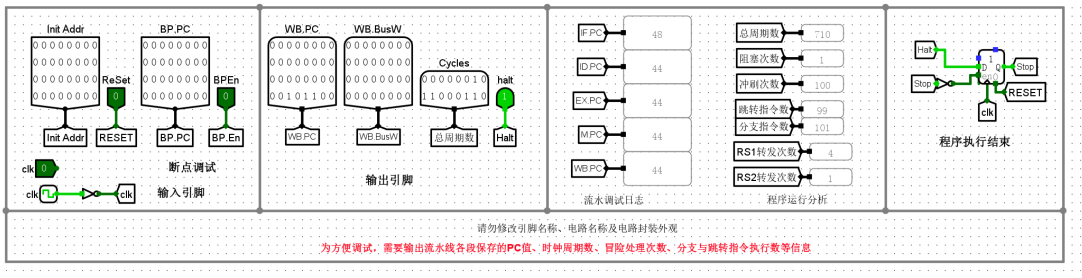


图 9: 累加和程序（lab8.2）执行性能（ $n = 100$ ，M 段跳转检测）

表 1: 累加和程序（lab8.2，M 段跳转检测）性能数据

性能指标	实测值
总周期数	710
阻塞次数	1
冲刷次数	100
跳转指令执行数	99
分支指令执行数	101
RS1 转发次数	4
RS2 转发次数	1

2.2 优化后的累加和程序 (lab8.2-1)

优化思路：原程序每轮循环需执行 bgeu (M 段预测失败时冲刷 3 周期) 和 jal (无条件跳转，必然冲刷 3 周期) 两条跳转相关指令。通过将”超出时跳出”改为”未超出时继续”，并将无条件 jal 合并到条件分支中，使循环过程中的 bgeu 大多向后跳转（预测不跳转 → 预测失败），但消除了每轮必然冲刷的 jal，使总冲刷次数由 100 次降至 99 次，并节省 99 条 jal 指令的执行开销。

优化后的汇编代码 lab8.2-1.asm 如下：

Listing 2: 优化后的累加和程序 lab8.2-1.asm

```

1  .text
2  main:
3      lw    a0, 0(x0)          # 从数据区 0x0 读取参数 n; n=0 则退出
4      beq   a0, x0, fail
5      ori   a2, x0, 1          # 循环变量 i = 1
6      xor   a3, a3, a3         # 累加和 S = 0
7  loop:
8      add   a3, a3, a2         # S = S + i
9      addi  a2, a2, 1          # i++
10     bgeu  a0, a2, loop       # n >= i 则继续循环 (删去了 jal)
11     sw    a3, 4(x0)         # 保存结果到 Mem[0x4]
12  pass:
13     lui   a0, 0xc10
14     addi  a0, a0, -18         # a0 = 0x00c0ffee
15     ecall
16  fail:
17     lui   a0, 0xdeade
18     addi  a0, a0, -339       # a0 = 0xdeaddead
19     ecall

```

执行结果如下：

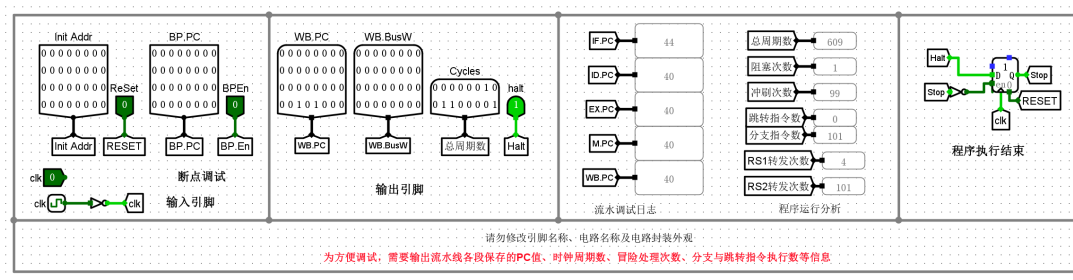


图 10: 优化后累加和程序 (lab8.2-1) 执行性能 ($n = 100$, M 段跳转检测)

表 2: 优化后累加和程序 (lab8.2-1, M 段跳转检测) 性能数据

性能指标	实测值
总周期数	609
阻塞次数	1
冲刷次数	99
跳转指令执行数	0
分支指令执行数	101
RS1 转发次数	4
RS2 转发次数	101

2.3 错误现象及分析

在完成本次实验的过程中，没有遇到任何问题。

3 实验三 结构优化设计

3.1 优化原理

在实验一/二中，跳转检测位于 M 段，预测失败时需冲刷 IF、ID、EX 三个流水段，代价为 **3 个时钟周期**。将跳转检测迁移至 EX 段后，仅需冲刷 IF、ID 两个流水段，代价降为 **2 个时钟周期**。

可行性分析：EX 段末尾 ALU 已完成运算并输出 Result 与 Zero 标志；旁路转发检测也在 EX 段完成，BusA/BusB 均已取得正确值。因此可在 EX 段末尾可靠地判断是否需要跳转。

3.2 电路修改

将跳转检测从 M 段迁移至 EX 段，主要修改如下：

- 跳转检测模块输入信号改为 EX.BandJ、EX.Zero 和 EX.Result[0]；
- PCsrc 在 EX 段产生，直接送至 IF 段 PC MUX；
- 冲刷信号 CLR 仅连接 IF/ID 和 ID/EX 两个流水段寄存器（原来为三个，去掉了 EX/M）；
- PCTarget 在 EX 段计算后直接送至 IF 段（可经 EX/M 寄存器旁路，也可直接连线）。

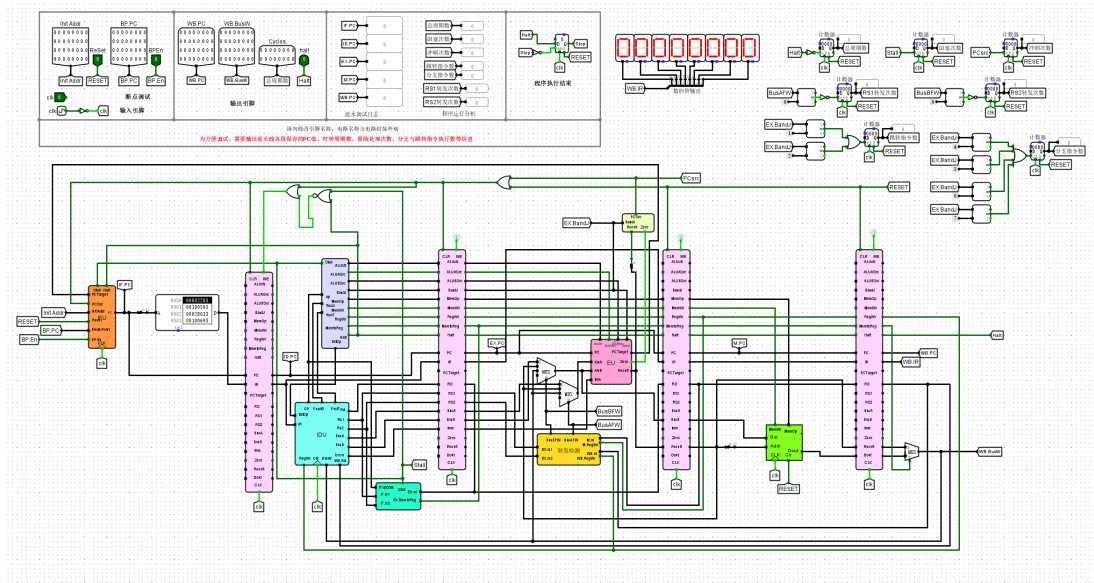


图 11: 跳转检测迁移至 EX 段后的流水线 CPU 顶层电路图 (lab8.3.circ)

3.3 仿真验证

3.3.1 累加和程序验证

在 lab8.3.circ 中加载 lab8.2-1.hex, 设置参数 $n = 100$, 执行程序:

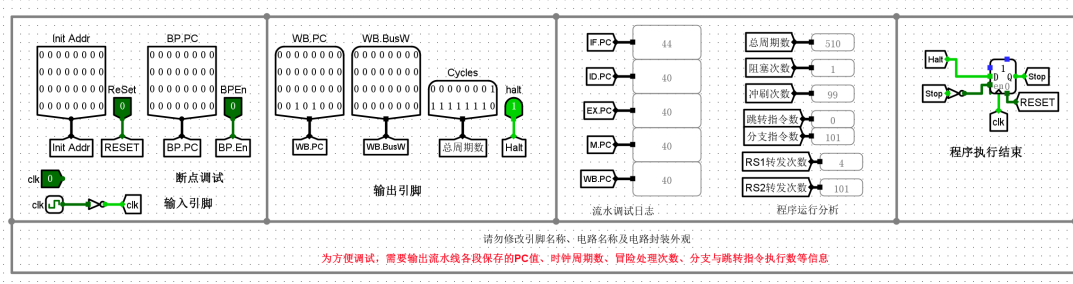


图 12: EX 段跳转检测优化后的累加和程序执行性能 ($n = 100$)

表 3: EX 段跳转检测优化后的累加和程序性能数据 ($n = 100$)

性能指标	实测值
总周期数	510
阻塞次数	1
冲刷次数	99
跳转指令执行数	0
分支指令执行数	101

续表：EX 段跳转检测优化后的累加和程序性能数据（ $n = 100$ ）

性能指标	实测值
RS1 转发次数	4
RS2 转发次数	101

性能提升分析：对于 $n = 100$ 的累加和程序，循环共发生 99 次分支预测失败（每次 bgeu 实际跳转而预测不跳转）。M 段检测时每次代价 3 周期，总损失 $99 \times 3 = 297$ 周期；EX 段检测时每次代价 2 周期，总损失 $99 \times 2 = 198$ 周期，因此总周期数减少约 99 个，与实验结果一致。

3.3.2 冒泡排序程序验证

在 lab8.3.circ 中加载 lab8.3.hex，在数据存储器中加载 lab8.3.ram3.txt - lab8.3.ram0.txt（小端序存储方式），执行程序：

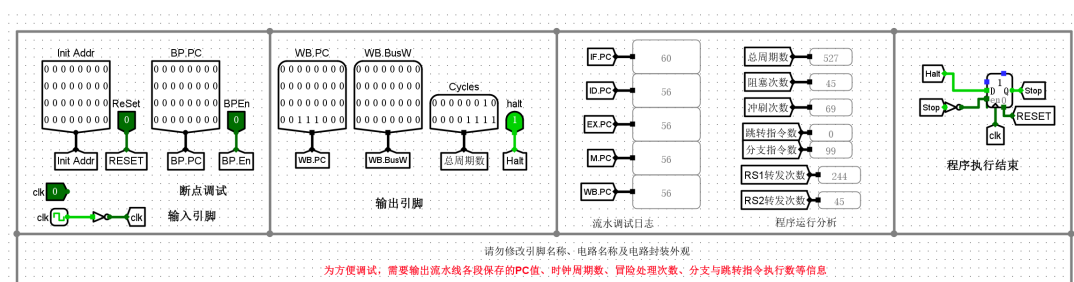


图 13: 冒泡排序测试程序执行性能（EX 段跳转检测，lab8.3.circ）

执行完毕后数据存储器中的数据已按照升序方式排序，运行成功。

3.4 错误现象及分析

在完成实验的过程中，没有遇到任何问题。

4 实验四 官方测试集验证

使用实验 8 提供的官方测试集（testcase.zip），在流水线 CPU（lab8.4.circ）中依次加载每条指令的测试文件，验证实验结果。

需要注意的是，该测试集中判断测试成功或失败的结束代码为 dead10cc（区别于实验一的 0x73 ecall），需在控制器中增加对该代码的识别逻辑来触发中止信号（Halt）。测试通过时 a0 寄存器值为 0x00c0ffee，测试失败时为 0xdeaddead。

表 4: RISC-V 官方测试集测试数据表

序号	测试指令	总周期数	阻塞次数	冲刷次数	跳转指令执行数	分支指令执行数	RS1转发次数	RS2转发次数
1	add	493	0	16	0	68	84	119
2	addi	252	0	7	0	37	53	46
3	and	513	0	16	0	57	158	91
4	andi	208	0	7	0	26	53	30
5	auipc	61	0	3	2	3	5	4
6	beq	341	0	27	0	72	37	58
7	bge	377	0	36	0	81	41	62
8	bgeu	402	0	36	0	81	66	60
9	blt	341	0	27	0	72	37	58
10	bltu	366	0	27	0	72	57	57
11	bne	345	0	29	0	72	38	58
12	jal	57	0	3	2	3	4	4
13	jalr	137	0	13	9	10	27	13
14	lb	257	2	7	0	37	79	54
15	lbu	257	2	7	0	37	79	54
16	lh	269	2	7	0	37	83	44
17	lhu	276	2	7	0	37	88	46
18	lui	63	0	1	0	6	4	9
19	lw	279	2	7	0	37	89	46
20	or	516	0	16	0	57	162	92
21	ori	215	0	7	0	26	60	31
22	sb	452	0	13	0	59	118	84
23	sh	505	0	13	0	59	144	88
24	sll	521	0	16	0	72	94	139
25	slli	251	0	7	0	37	53	51
26	slt	487	0	16	0	68	79	120
27	slti	247	0	7	0	37	48	53
28	sltiu	247	0	7	0	37	48	54
29	sltu	487	0	16	0	68	79	120
30	sra	540	0	16	0	73	116	141
31	srai	266	0	7	0	37	73	55

续下页

续表：RISC-V 官方测试集测试数据表

序号	测试指令	总周期数	阻塞次数	冲刷次数	跳转指令执行数	分支指令执行数	RS1 转发次数	RS2 转发次数
32	srl	534	0	16	0	73	102	141
33	srli	260	0	7	0	37	63	57
34	sub	485	0	16	0	67	82	119
35	sw	512	0	13	0	59	153	86
36	xor	515	0	16	0	57	161	92
37	xori	217	0	7	0	26	65	31

5 思考题

5.1 转发检测未判断操作码的影响

可能的影响：不检查 opcode 时，对于不使用 rs1/rs2 的指令（如 LUI、AUIPC、JAL），其指令编码中的 rs1/rs2 字段可能恰好与流水线中某条指令的 rd 相同，转发检测模块会错误地将 BusAFW 或 BusBFW 置为非零，对无需转发的数据通路进行不必要的转发。

是否导致程序执行错误：通常不会导致错误。以 LUI 为例，其 ALUASrc 使 ALU 的 A 端实际接 PC（而非 BusA），ALUBSrc 选择立即数，运算结果仅取决于立即数，与 BusA 无关；类似地，不需要 rs2 的指令（I 型、U 型等）中 ALUBSrc = 10（选立即数），错误转发的 BusB 数据同样不影响结果。因此即使产生了不必要的转发信号，最终送入 ALU 的操作数仍然正确。

解决方案及必要性：可在转发检测逻辑中加入对 EX 段当前指令 opcode 的判断，仅当指令确实使用 rs1 或 rs2 时才生成对应转发信号。由于不影响功能正确性，在本实验设计中并非必要，但在追求功耗或安全性优化时可以加入。

被阻塞的指令能否产生有效转发信号：不能。阻塞时 ID 段控制信号全部清零，该指令在进入 EX 段时 RegWr=0，转发检测模块因此不将其视为有效数据生产者，不产生有效转发信号。气泡（Bubble）在各流水段寄存器中传递时，其 RegWr=0 或 rd=x0，均不满足转发条件，不会干扰后续指令的转发逻辑。

5.2 跳转检测迁移至 ID 段的可行性分析

理论上可以将跳转检测进一步迁移至 ID 段，使预测失败惩罚从 2 周期降至 1 周期，但面临以下主要挑战：

1. **数据冒险：**ID 段读取的 BusA/BusB 可能尚未包含最新值（前序指令的结果仍在

EX/M/WB 段流动)，必须在 ID 段增加专用的转发路径，或额外阻塞等待数据就绪，显著增加设计复杂度。

2. **比较逻辑前移：**条件分支需在 ID 段完成操作数比较（BusA vs BusB），而此时可能有多级数据冒险未解决，需配合 ID 段转发或阻塞。
3. **JALR 的复杂性：**JALR 的跳转目标依赖 rs1 的值，rs1 在 ID 段读取时可能存在 RAW 冒险，需转发支持，逻辑更加复杂。
4. **时序压力：**ID 段已包含译码、立即数扩展、读寄存器等操作，再加入比较和跳转判断逻辑，可能使 ID 段成为关键路径，进而限制整个流水线的时钟频率。

综合来看，对于无条件跳转 JAL（目标仅由 PC+imm 决定），迁移至 ID 段相对可行；对于条件分支，迁移到 ID 段实现代价较高，需在性能收益与设计复杂度之间权衡。

5.3 三种累加和程序执行性能分析

以参数 $n = 100$ （0x64）的实测数据进行分析：

表 5: 三种情形下累加和程序执行性能对比（ $n = 100$ ）

性能指标	lab8.2 (M 段 + jal)	lab8.2-1 (M 段, 无 jal)	lab8.3 (EX 段, 无 jal)
总周期数	710	609	510
阻塞次数	1	1	1
冲刷次数	100	99	99
跳转指令执行数	99	0	0
分支指令执行数	101	101	101
RS1 转发次数	4	4	4
RS2 转发次数	1	101	101

数据准确性讨论：

- **总周期数：**由硬件计数器直接统计，数据准确。
- **阻塞次数：**准确，为 1。首条 `lw a0,0(x0)` 之后紧跟使用 a0 的 `beq a0,x0,fail`，产生一次 load-use 阻塞，此后循环体中无 load-use 冒险。
- **冲刷次数：**准确。lab8.2 中每轮循环的 `jal`（必然冲刷）和 `bgeu`（不跳转时冲刷，即循环结束时）各一次，共 $99 + 1 = 100$ 次；lab8.2-1/lab8.3 删去 `jal` 后，99 次 `bgeu` 向后跳转均预测失败，共 99 次冲刷。

- **RS2 转发次数差异**: lab8.2 中 `bgeu a2,a0,finish` 的 rs2 为 a0 (由首次 `lw` 写入, 后续无 RAW), 故 RS2 转发次数极少 (仅 1 次); lab8.2-1 中 `bgeu a0,a2,loop` 的 rs2 为 a2, 而 a2 由上一条 `addi a2,a2,1` 更新, 每次循环均存在 RAW 冒险, 故 RS2 转发次数为 101 次。

- **分支/跳转指令执行数**: 准确, 由指令类型识别计数器统计。

平均 CPI 计算 (以 lab8.2-1, M 段, $n = 100$ 为例):

程序动态执行指令数估算: 前置 4 条 (`lw, beq, ori, xor`) + 循环体 3 条/轮 $\times 100$ 轮 (`add, addi, bgeu`) + 结尾约 4 条 (`sw, lui, addi, ecall`) ≈ 308 条。

$$\text{平均 CPI} \approx \frac{609}{308} \approx 1.98$$

CPI 超出 1 的主要来源: 99 次分支预测失败 $\times 3$ 周期冲刷代价 = 297 周期额外损失; 1 次 load-use 阻塞 = 1 周期。

提升执行效率的方法:

1. **程序优化 (软件层面)**: 如实验二所示, 删除冗余无条件跳转, 减少分支数量, 降低冲刷代价。
2. **提前跳转检测 (硬件层面)**: 如实验三, 将跳转检测迁移至 EX 段, 每次冲刷代价从 3 降为 2 周期, 总周期数减少约 99 个。
3. **循环展开**: 将循环体展开 k 次, 分支频率降为原来的 $1/k$, 减少冲刷总次数, 但代码体积增大。
4. **动态分支预测**: 对循环类跳转引入 1 位或 2 位动态预测器, 预测准确率接近 100%, 99 次冲刷可降至约 1 次 (仅循环退出时失败)。

5.4 实现非法指令与结果溢出的异常处理

异常检测:

- **非法指令**: 在 ID 段控制器中, 当 `opcode/funct3/funct7` 不匹配任何合法指令时, 输出非法指令标志信号 `IllegalOp`, 随流水段寄存器传递至 WB 段。
- **结果溢出**: 在 EX 段 ALU 中, 对带符号加减法增加溢出检测 (两同号数相加结果异号, 或等效的减法情形), 输出 `Overflow` 标志, 同样传至 WB 段。

异常类型标识: 使用 2 位异常码随流水段同步传递: 00= 无异常, 01= 非法指令, 10= 溢出, 11= 保留。

异常处理流程: 当异常信号到达 WB 段时, 执行以下操作:

1. 触发冲刷信号, 清除流水线中所有后续指令;

2. 将异常发生时的 PC 保存到专用寄存器（如 EPC）；
3. 将异常类型码写入状态寄存器（如 mcause），标识异常类型；
4. PC 跳转至对应异常处理程序的入口地址（异常向量表）；
5. 异常处理完毕后，通过专用返回指令（如 RISC-V 的 `mret`）恢复执行：溢出等可恢复异常可返回异常指令的下一条继续执行，非法指令通常终止程序。

5.5 1 位动态分支预测的实现

1 位动态预测器为每条分支指令维护 1 位历史（上次是否跳转），预测当前行为与上次一致：

1. **分支历史表（BHT）**：以 PC 若干低位为索引，每个条目存储 1 位（上次跳转 =1，不跳转 =0）。
2. **IF 阶段查表预测**：取指时以 PC 低位索引 BHT，得到预测结果 `PredTaken`；若为 1，则 PC 更新为预测跳转目标（`PC+imm`，需在 ID 阶段简单计算后反馈）；否则顺序执行。
3. **EX 阶段验证**：完成实际跳转判断，若结果与预测不一致则冲刷流水线（2 周期损失），并将 PC 修正为正确地址。
4. **BHT 更新**：用 EX 段实际跳转结果更新 BHT 对应条目。

对于累加和这类单方向循环，1 位预测器仅在第一次执行（冷启动）和循环退出时各失败一次，共 2 次预测失败，相比当前静态“预测不跳转”的 99 次失败有显著提升。

5.6 高级流水线设计方法

高级流水线设计方法主要包括以下几类：

1. **超标量（Superscalar）**：每个时钟周期同时取指、译码并执行多条无相关指令，设置多套执行单元，理论 CPI 小于 1，需要复杂的相关检测与指令调度逻辑。
2. **乱序执行（Out-of-Order, OOO）**：通过保留站（Reservation Station）和重排序缓冲区（ROB）实现，允许后续无相关指令绕过正在等待数据的指令先行执行，充分发掘指令级并行（ILP）。结合寄存器重命名消除 WAR、WAW 伪相关。
3. **高级动态分支预测**：采用两级相关预测（如 GShare）、混合预测（Tournament Predictor）或 TAGE 等算法，将分支预测准确率提升至 95% 以上，大幅降低控制冒险代价。

4. **深度流水线**：将流水段细分为更多级（例如 Intel Pentium 4 的 31 级），降低各级组合逻辑复杂度，提高时钟主频。代价是预测失败惩罚更大，需配合精确动态预测器。
5. **超长指令字（VLIW）**：由编译器静态分析并行性，将多条可并行执行的操作打包为超长指令字，硬件无需动态调度，实现简单、功耗低，但对编译器依赖程度高，代码可移植性差。
6. **向量/SIMD 扩展**：针对数据并行应用，以宽数据通路一次对多个数据元素执行相同运算（如 RISC-V 的 V 扩展），大幅提升数据级并行（DLP）效率，适合多媒体处理、科学计算等场景。